APPENDIX – II

SOURCE CODE IMPLEMENTATION OF CORE PROJECT MODULES

This appendix documents the essential implementation logic of the VoltFleet AI platform across its six primary architectural modules described in Chapter 6. Rather than reproducing peripheral boilerplate, each listing highlights the core computational algorithms, mathematical formulations, machine learning regression pipelines, and secure telematics ingestion routines that govern system behavior.

Listing 1: Real-Time Fleet Telemetry & Battery State Physics Simulator [Module 1: Fleet Command Center]

Source File: `backend/simulator.py`

Executes background multithreaded simulation of vehicle operational cycles, computing dynamic battery SOC drain, DC fast-charging voltage variations, inverter thermal response, and critical battery transitions.

```
def simulate_vehicle_telemetry(db: Session, vehicle: Vehicle, now: datetime):
    """Simulate physics-grounded telematics state transitions for an EV unit."""
    # 1. State: Charging at Depot (DC Fast Charge Dynamic)
    if vehicle.status == 'charging':
        charge_rate = random.uniform(3.0, 6.0)  # SOC % increase per tick
        vehicle.current_soc = min(100.0, vehicle.current_soc + charge_rate)
        db.add(BatteryLog(
            time=now, vehicle_id=vehicle.vehicle_id, battery_percentage=round(vehicle.current_soc, 2),
            voltage=380.0 + random.uniform(-4, 4), current_draw=-45.0,  # Negative indicates charging
            temperature_celsius=34.0 + random.uniform(-1, 2), soh_percentage=round(vehicle.battery_health, 2)
        ))
        if vehicle.current_soc >= 100.0: vehicle.status = 'idle'

    # 2. State: Driving in Mission (Aerodynamic & Velocity Energy Drain)
    elif vehicle.status == 'driving':
        speed = max(15.0, min(110.0, vehicle.current_speed + random.uniform(-6, 6)))
        vehicle.current_speed = round(speed, 1)
        drain_rate = 0.45 + (speed / 100.0) * 0.75 + random.uniform(0.05, 0.20)
        vehicle.current_soc = max(0.0, vehicle.current_soc - drain_rate)
        db.add(SpeedLog(time=now, vehicle_id=vehicle.vehicle_id, speed_kmh=vehicle.current_speed))
        db.add(BatteryLog(
            time=now, vehicle_id=vehicle.vehicle_id, battery_percentage=round(vehicle.current_soc, 2),
            voltage=370.0 - (100.0 - vehicle.current_soc) * 0.35, current_draw=25.0 + (speed * 0.4),
            temperature_celsius=29.0 + (speed / 35.0), soh_percentage=round(vehicle.battery_health, 2)
        ))
        if vehicle.current_soc <= 15.0: vehicle.status = 'charging'  # Low-SOC auto-reroute
```

Listing 2: Role-Based Cryptographic Access Control & Token Ingestion [Module 2: Role-Based Authentication]

Source File: `backend/auth.py`

Enforces JWT token validation, role-based authorization guards (distinguishing Fleet Managers from Drivers), and encrypted credential verification for all telematics and operational endpoints.

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/api/auth/login')

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """Generates signed cryptographically secure JSON Web Token (JWT)."""
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({'exp': expire})
    return jwt.encode(to_encode, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)

def verify_token_and_get_user(token: str, db: Session) -> dict:
    """Decodes JWT, validates signature, and resolves Manager or Driver role permissions."""
    exc = HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail='Invalid authentication
credentials')
    try:
        payload = jwt.decode(token, settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM])
        user_id, role = payload.get('id'), payload.get('role')
        if not user_id or not role: raise exc
    except jwt.PyJWTError: raise exc
    if role == 'manager':
        user = db.query(Admin).filter(Admin.admin_id == user_id).first()
        return {'id': user.admin_id, 'email': user.email, 'role': 'manager'} if user else (_ for _ in
()).throw(exc)
    elif role == 'driver':
        user = db.query(Driver).filter(Driver.driver_id == user_id).first()
        return {'id': user.driver_id, 'email': user.email, 'role': 'driver'} if user else (_ for _ in
()).throw(exc)
    raise exc
```

Listing 3: Dynamic Multi-Stop Route Optimizer & EV Energy Consumption Model [Module 3: Route Optimizer]

Source File: `backend/routers/route_optimizer.py`

Calculates geodetic Haversine distances, road routing profiles via OSRM, and models commercial EV fleet energy expenditure (kWh) and battery SOC depletion percentage per mission.

```
def calculate_haversine_distance(lat1: float, lng1: float, lat2: float, lng2: float) -> float:
    """Calculate great-circle distance on spherical Earth in kilometers."""
    R_KM = 6371.0
    d_lat, d_lng = math.radians(lat2 - lat1), math.radians(lng2 - lng1)
    a = math.sin(d_lat / 2)**2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(d_lng
/ 2)**2
    return R_KM * (2 * math.atan2(math.sqrt(a), math.sqrt(1 - a)))

def optimize_ev_route(start_coords, dest_coords, battery_cap_kwh=75.0):
    """Calculates road distance, drive duration, EV energy (kWh), and battery usage %."""
    osrm = fetch_osrm_route(start_coords[0], start_coords[1], dest_coords[0], dest_coords[1])
    if osrm and len(osrm.get('path_coordinates', [])) > 5:
        dist_km, duration_mins = osrm['distance_km'], osrm['duration_mins']
        path_coords = osrm['path_coordinates']
    else:
        dist_km = max(calculate_haversine_distance(start_coords[0], start_coords[1], dest_coords[0],
dest_coords[1]) * 1.25, 5.0)
        duration_mins = int((dist_km / 80.0) * 60)
        path_coords = generate_geodesic_curved_path(start_coords[0], start_coords[1], dest_coords[0],
dest_coords[1])
    energy_kwh = dist_km * 0.195  # EV efficiency model (~0.195 kWh/km commercial average)
    battery_usage_pct = min(round((energy_kwh / battery_cap_kwh) * 100.0, 1), 100.0)
    return {'distance_km': round(dist_km, 2), 'duration_mins': duration_mins,
            'energy_kwh': round(energy_kwh, 2), 'battery_usage_pct': battery_usage_pct, 'path': path_coords}
```

Listing 4: Machine Learning AI Driver Performance Scoring Model [Module 4: Driver Analytics]

Source File: `backend/ml_model.py & backend/routers/drivers.py`

Aggregates 7 operational driver telematics features (speed discipline, harsh braking, acceleration volatility, battery retention, and revenue), synthesizing them into an AI performance score.

```
def compute_driver_features(driver_id: str, db: Session) -> dict:
    """Extract 7 operational telematics dimensions from real-time database logs."""
    trips = db.query(TripLog).filter(TripLog.driver_id == driver_id).all()
    days = max(1, len(set(t.time.date() for t in trips))) if trips else 1
    total_dist = sum(t.distance_covered_km for t in trips) if trips else 0.0
    overspeed = sum(t.overspeeding_events_count for t in trips) if trips else 0
    speeds = db.query(SpeedLog.speed_kmh).filter(SpeedLog.driver_id == driver_id).all()
    avg_speed = sum(s[0] for s in speeds) / len(speeds) if speeds else 50.0
    socs = db.query(DriverRealtimeData.current_battery_pct).filter(DriverRealtimeData.driver_id ==
driver_id).all()
    eod_soc = sum(s[0] for s in socs) / len(socs) if socs else 45.0
    revs = db.query(RevenueLog.amount).filter(RevenueLog.driver_id == driver_id).all()
    return {'dist': total_dist / days, 'spd': avg_speed, 'hb': min(10.0, (overspeed * 0.40) / days),
            'ha': min(10.0, (overspeed * 0.35) / days), 'soc': eod_soc, 'rev': sum(r[0] for r in revs) / days
if revs else 150.0}

def predict_driver_score(driver_id: str, db: Session) -> float:
    """Predicts multi-pillar composite driver performance score (0-100) via ML."""
    model = get_performance_model()  # Multi-linear regression model
    f = compute_driver_features(driver_id, db)
    X = [[f['dist'], f['spd'], f['hb'], f['ha'], f['soc'], f['rev'], 5.0]]
    pred = float(model.predict(X)[0])
    return round(max(0.0, min(100.0, pred)), 1)
```

Listing 5: Physics-Informed ML EV Range Predictor & Training Pipeline [Module 5: Range & Battery Analytics]

Source File: `train_range_model.py & backend/ml_model.py`

Implements multi-capacity data augmentation, terrain/route encoding, and the Random Forest regression pipeline predicting real-time remaining EV driving range based on instantaneous telematics, HVAC load, and topography.

```
def train_and_export_range_model(csv_path='driver_realtime_data.csv'):
    """Trains physics-informed Random Forest EV Range Regressor across multi-capacity fleet."""
    df = pd.read_csv(csv_path)
    route_map, elev_map = {'City': 0, 'Mixed': 1, 'Highway': 2}, {'Downhill': -1, 'Flat': 0, 'Uphill': 1}
    df['route_type_encoded'] = df['route_type'].map(lambda x: route_map.get(x, 1))
    df['elevation_encoded'] = df['elevation_ahead'].map(lambda x: elev_map.get(x, 0))
    augmented = [df.assign(battery_capacity_kwh=c, actual_remaining_range_km=(df['actual_remaining_range_km']
* (c / df['battery_capacity_kwh'])).round(1))
                 for c in [17.3, 30.0, 40.0, 50.3, 59.0, 75.0]]
    df_all = pd.concat(augmented, ignore_index=True)
    features = ['current_battery_pct', 'battery_capacity_kwh', 'current_speed_kmh',
                'ac_status', 'outside_temperature_c', 'route_type_encoded', 'elevation_encoded']
    rf = RandomForestRegressor(n_estimators=100, max_depth=16, min_samples_split=4, random_state=42)
    rf.fit(df_all[features], df_all['actual_remaining_range_km'])
    return rf, route_map, elev_map

def predict_range(soc, speed, ac, temp, route_type, elev, cap_kwh, model, r_map, e_map):
    """Real-time remaining range inference incorporating thermal HVAC and elevation."""
    if soc <= 0.0: return 0.0
    vec = [[soc, cap_kwh, speed, ac, temp, r_map.get(route_type, 1), e_map.get(elev, 0)]]
    return max(0.0, round(float(model.predict(vec)[0]), 1))
```

Listing 6: Smart Time-of-Use Charging Dispatcher & Predictive Maintenance Planner [Module 6: Smart Charging & Maintenance]

Source File: `backend/routers/charging.py & backend/routers/maintenance.py`

Implements automated Time-of-Use (TOU) off-peak charging schedule optimization to minimize fleet electricity costs, while evaluating battery degradation metrics to trigger proactive maintenance work orders.

```python
def schedule_offpeak_charging(vehicles: List[Vehicle]) -> List[dict]:
    """Schedules depot charging allocations aligned with off-peak Time-of-Use (TOU) tariffs."""
    schedule, slot = [], 1
    for v in vehicles:
        if v.status in ['charging', 'idle'] or v.current_soc < 70.0:
            hour = (22 + (slot // 3)) % 24  # Staggered night-time off-peak schedule (22:00 to 05:00)
            rate = '150 kW' if (not v.motor_power_kw or v.motor_power_kw >= 100) else '120 kW'
            schedule.append({
                'schedule_id': f'SCH-{v.vehicle_id}', 'time': f'{hour:02d}:{(slot % 3) * 30:02d}',
                'depot': f'Depot {"A" if slot % 2 == 0 else "B"} — Bay {((slot - 1) % 4) + 1}',
                'vehicle_id': v.vehicle_id, 'duration': v.charging_time_spec or '4h', 'rate': rate
            })
            slot += 1
    return schedule

def evaluate_predictive_maintenance(vehicles: List[Vehicle]) -> List[dict]:
    """Generates condition-based maintenance actions triggered by battery State of Health (SOH)."""
    work_orders = []
    for v in vehicles:
        if v.battery_health < 75.0:
            work_orders.append({'vehicle_id': v.vehicle_id, 'severity': 'CRITICAL',
                                'action': 'Urgent Battery Cell Balancing & Thermal Pack Overhaul', 'sla': '2
days'})
        elif v.battery_health < 88.0:
            work_orders.append({'vehicle_id': v.vehicle_id, 'severity': 'WARNING',
                                'action': 'Battery Impedance Diagnostics & Inverter Service', 'sla': '5
days'})
    return work_orders
```

System Implementation Environment & Module Verification

Backend Framework & API Engine	FastAPI (Python 3.11) with Uvicorn Asynchronous ASGI Server
Machine Learning & Range Regression	Scikit-Learn 1.3.2 (RandomForestRegressor, Multi-Linear Regression)
Database & ORM Layer	SQLite / PostgreSQL with SQLAlchemy 2.0 & Pydantic Data Validation
Security & Telematics Encryption	Bcrypt password hashing & PyJWT for Role-Based Access Control (RBAC)